

Software Requirements Specification (SRS)

Sala Katoliki / Catholic Prayer Mobile Application

MVP Version - Flutter Implementation

Document Attribute	Value
Product Name	Sala Katoliki / Catholic Prayer
Document Type	Software Requirements Specification (SRS)
Prepared For	Kilimanjaro Technology
Developer / Company	Kilimanjaro Technology
Technology Stack	Flutter, Dart, bundled local JSON content, local storage
Version	1.0 - MVP SRS
Date	2 June 2026
Status	Development-ready baseline

Confidentiality: Internal project document for planning, development, QA, and release execution.

Document Control

Version	Date	Author / Owner	Description
1.0	2 June 2026	Kilimanjaro Technology	Initial MVP SRS for Flutter mobile application.

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 Assumptions and Dependencies
- 3. External Interface Requirements
 - 3.1 User Interfaces
 - 3.2 Hardware Interfaces
 - 3.3 Software Interfaces
 - 3.4 Communications Interfaces
- 4. Functional Requirements
 - 4.1 System Workflows
 - 4.2 Feature Matrices
 - 4.3 Input/Output Handling
 - 4.4 User Stories and Use Cases
- 5. Non-Functional Requirements
 - 5.1 Performance
 - 5.2 Reliability and Availability
 - 5.3 Security
 - 5.4 Maintainability and Scalability
 - 5.5 Portability
 - 5.6 Mobile Non-Functional Requirements
 - 5.7 Compliance and Deployment
- 6. Supporting Information
 - 6.1 Data Models
 - 6.2 Analysis Models
 - 6.3 Acceptance Criteria

- 6.4 Traceability Matrix
- Appendix A. Recommended Content File Structure
- Appendix B. Out-of-Scope Backlog

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional, non-functional, interface, data, mobile, compliance, and deployment requirements for the MVP of Sala Katoliki / Catholic Prayer, a Flutter-based mobile application developed by Kilimanjaro Technology. The document is intended for product owners, Flutter developers, QA engineers, UI/UX designers, content contributors, and future maintainers.

The software is being built to solve the following problems:

- Catholic users need quick access to essential prayers, Rosary guidance, and novenas without carrying printed prayer books.
- Many users need prayer resources in a language they understand, especially English and Kiswahili.
- Users may have unreliable or costly internet access, so core prayer features must work offline.
- The project needs a maintainable content structure so new prayers can be added later without creating new hard-coded screens.
- The open-source nature of the project requires clear data structures, contribution rules, and content attribution.

1.2 Scope

The MVP scope is a standalone mobile application using Flutter. The application shall not require account registration or authentication. Prayer content shall be bundled locally in structured JSON files for the MVP, while the application architecture shall support future migration to remote content delivery or a CMS without rewriting the UI.

1.2.1 In Scope

- First-launch language selection: English and Kiswahili.
- Home / Today screen with quick access to high-value prayer actions.
- Common Catholic prayer library.
- Interactive Rosary with mysteries by day and step-by-step prayer flow.
- Novena module with 9-day progress tracking.
- Favorites, offline search, and local reminder notifications.
- Settings for language, reminder, font size, theme, About, and content sources.
- Local bundled content system using JSON files and dynamic rendering.
- Android-first release with iOS support planned within the same Flutter codebase.

1.2.2 Out of Scope for MVP

- Authentication, account registration, profile management, or social sign-in.
- Cloud sync, user accounts, admin dashboard, or real-time backend content management.
- Payment, donations, subscriptions, ads, or monetization.
- Audio prayers, video content, podcasts, or guided meditation library.

- Community posting, chat, comments, or parish social networking.
- Full Bible module, full Catechism module, Mass booking, parish management, and sacrament scheduling.
- AI spiritual advisor or automated pastoral guidance.
- Use of copyrighted Bible translations, Mass readings, or devotional material without written permission or valid licensing.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification; the formal document describing what the software must do and how it must perform.
MVP	Minimum Viable Product; the smallest product release that delivers meaningful value and validates product direction.
Sala Katoliki	Kiswahili product name meaning Catholic Prayer.
Prayer Content	Structured devotional text including prayer title, body, category, language, source, and metadata.
Bundled Content	JSON content files packaged inside the mobile app at release time.
Dynamic Rendering	The UI reads content data and renders screens automatically instead of creating one screen per prayer.
Rosary	Catholic devotional prayer using mysteries and repeated prayers across decades.
Novena	A devotional prayer usually prayed for nine days.
Offline-first	Core features operate without internet access.
Flutter	Google open-source UI toolkit used to build the application with Dart.
Dart	Programming language used by Flutter.
API	Application Programming Interface.
JSON	JavaScript Object Notation; structured text format used for bundled content files.
QA	Quality Assurance.
RBAC	Role-Based Access Control; not required for MVP because no authentication/admin module exists.
PII	Personally Identifiable Information.
L10n	Localization; adapting UI and content to a language/locale.
i18n	Internationalization; preparing the software to support multiple languages.

1.4 References

Reference	Source / URL	Use in this SRS
Flutter supported platforms	https://docs.flutter.dev/reference/supported-platforms	Confirms Flutter supports mobile deployment targets including Android and iOS.
Flutter iOS deployment documentation	https://docs.flutter.dev/deployment/ios	Used for iOS deployment consideration; Flutter documentation states Flutter supports iOS 13 and later.
Apple App Store Review Guidelines	https://developer.apple.com/app-	Compliance baseline for iOS App

	store/review/guidelines/	Store release.
Apple App Review distribution guidance	https://developer.apple.com/distribute/app-review/	Requires functional links, support contact, and privacy policy for apps submitted to review.
Google Play Developer Policy Center	https://play.google/developer-content-policy/	Compliance baseline for Google Play release.
Google Play target API level requirement	https://developer.android.com/google/play/requirements/target-sdk	Used for Android target SDK compliance planning.
Android app compatibility documentation	https://developer.android.com/guide/app-compatibility	Used for Android API and device compatibility planning.
Internal product decision	Conversation decision: use bundled local JSON content for MVP and design future remote content support.	Primary product architecture decision for prayer content management.

2. Overall Description

2.1 Product Perspective

Sala Katoliki is a new standalone Flutter mobile application, not an update to an existing platform and not a subsystem of a larger enterprise system. The MVP shall be built as an offline-first mobile product with local bundled content and local device storage for preferences, favorites, Rosary progress, and novena progress.

The MVP architecture is intentionally simple:

Flutter Mobile App -> Local Content Loader -> Bundled JSON Content -> Dynamic UI Rendering -> Local Device Storage

Future architecture shall allow:

Flutter Mobile App -> Bundled Default Content + Optional Remote Content Updates -> Local Cache

2.2 Product Functions

- Language selection and localization.
- Home / Today prayer hub.
- Prayer category browsing and prayer detail reading.
- Favorites and search.
- Interactive Rosary.
- Novena browsing, starting, progress tracking, and continuation.
- Daily local reminder notifications.
- Offline content availability.
- Settings and About/content source information.
- Developer-friendly content file system for adding new prayers later.

2.3 User Classes and Characteristics

User Class	Description	Technical Skill	Access Privilege	Operational
------------	-------------	-----------------	------------------	-------------

				Frequency
General Catholic User	User who needs everyday Catholic prayers and devotional resources.	Low to medium	Can view prayers, set preferences, save favorites, track progress.	Daily or weekly
New Catholic / Learner	User learning common prayers, Rosary sequence, or novenas.	Low	Can use guided flows and read explanatory content.	Several times per week
Parish / Youth Group User	User who may use the app in group prayer or catechism sessions.	Low to medium	Can browse prayers quickly; no admin rights.	Weekly or event-based
Multilingual User	User who prefers English or Kiswahili.	Low	Can switch app language and view available localized content.	Daily or weekly
Content Contributor	Developer or trusted editor adding prayer JSON files through repository workflow.	Medium to high	No in-app admin rights for MVP; contributes via code/content repository.	Occasional
QA Engineer	Tester validating requirements against builds.	Medium	Test access only; no special in-app privilege.	During development and release cycles

2.4 Operating Environment

2.4.1 Mobile Operating Environment

Environment Item	MVP Requirement
Mobile Framework	Flutter stable channel with Dart.
Primary Platform	Android first.
Secondary Platform	iOS supported by Flutter codebase; release may follow Android after QA.
Android Minimum OS	Android 8.0 / API level 26 minimum recommended for broader compatibility. The app shall target the current Google Play target API requirement at build/release time.
Android Target SDK	At release time, target SDK shall comply with current Google Play requirements. As of the referenced Google Play requirement, existing apps must target Android 14 / API 34 or higher for availability to new users on newer Android devices.
iOS Minimum OS	iOS 13+ recommended because Flutter documentation states Flutter supports iOS 13 and later.
Minimum RAM	2 GB RAM recommended; app shall be optimized for low-end Android devices.
Storage Requirement	Initial installed app should target under 80 MB where practical; bundled content should be text-first and lightweight.
Network	Internet is not required for core MVP features. Optional future content update check may use HTTPS.

Required Sensors	No GPS, camera, microphone, NFC, Bluetooth, biometrics, or background location required.
Screen Support	Minimum supported logical width: 320 dp. App must support common phone screens and responsive scaling.

2.5 Design and Implementation Constraints

- The mobile application shall be implemented using Flutter and Dart.
- The MVP shall not include authentication, social login, biometric login, or account-based profile features.
- Prayer content shall not be embedded directly inside UI screens. Content shall be stored in structured assets such as JSON.
- The UI shall dynamically render prayer, category, Rosary, and novena screens based on content metadata.
- The MVP shall work offline for common prayers, Rosary, novenas, favorites, search, settings, and reminders.
- Sensitive permissions such as camera, microphone, contacts, precise location, HealthKit, Google Fit, Apple Pay, Google Pay, Bluetooth, and NFC shall not be requested.
- The app shall avoid copyrighted content unless Kilimanjaro Technology has verified permission or licensing.
- The app shall follow a clean architecture pattern such as presentation, domain, data/content, and local storage layers.
- State management shall be explicit and testable. Recommended options: Riverpod, Bloc/Cubit, Provider, or another team-approved Flutter state management library.
- The app shall use local notifications only for reminders and shall not use push notifications in MVP.

2.6 Assumptions and Dependencies

ID	Assumption / Dependency	Impact if False	Mitigation
AD-001	Kilimanjaro Technology will provide or approve initial prayer content in English and Kiswahili.	MVP content completeness may be reduced.	Allow missing translations and fallback behavior.
AD-002	Traditional prayer texts used in MVP are public domain or have permission.	Legal/content risk.	Maintain source metadata and review content before release.
AD-003	Flutter plugins for local storage and local notifications remain available and maintained.	Implementation delays.	Choose widely used packages and isolate plugin dependencies.
AD-004	Google Play and Apple App Store policies may change before release.	Release rejection or extra compliance work.	Re-check official policies during release preparation.
AD-005	No authentication is required for MVP.	If user accounts become mandatory, scope expands.	Keep user data local and delay sync/account features.

AD-006	Users benefit more from offline text prayer content than from audio/video in MVP.	Lower engagement for users expecting audio.	Validate with feedback and prioritize audio later if needed.
--------	---	---	--

3. External Interface Requirements

3.1 User Interfaces (UI)

The UI shall be calm, readable, respectful, and optimized for prayer. The app shall use a simple bottom navigation structure: Today, Pray, Novenas, Library. The design shall support light and dark mode, scalable fonts, and responsive layouts for small and large phone screens.

UI Requirement	Specification
Navigation	Bottom navigation with four main tabs: Today, Pray, Novenas, Library.
Touch targets	Primary touch targets shall be at least 48 x 48 dp where possible.
Typography	Prayer text must be readable with adjustable font size. Default body text should be at least 16sp equivalent.
Dark/Light Mode	Support system theme; manual theme override is recommended.
Gestures	Use standard mobile gestures: tap, scroll, back navigation. Do not require complex gestures for core prayer actions.
Responsive Scaling	Layouts must support 320 dp width minimum and common Android/iOS phone sizes.
Error UI	Errors must be understandable and non-technical, e.g., "This prayer is not available in the selected language yet."
Accessibility	Support screen reader labels for buttons, dynamic text scaling where practical, high contrast, and clear focus order.

3.1.1 MVP Screen Inventory

Screen	Purpose	Primary Actions
Splash	Initial brand display while app initializes.	Open app.
Language Selection	First launch language setup.	Choose English or Kiswahili.
Today	Daily entry point.	Open daily prayer, continue novena, start Rosary, open favorites.
Pray	Browse prayer categories.	Open category, search, open prayer.
Prayer Detail	Read a prayer.	Favorite, share/copy optional, adjust font size.
Rosary Home	Start or continue Rosary.	Choose mystery or use mystery of the day.
Rosary Step	Guided Rosary sequence.	Next, previous, restart, exit and save progress.
Novenas	Browse novenas.	Start or continue novena.
Novena Detail	Read novena days and track	Mark day complete, set reminder

	progress.	optional.
Search	Find prayers across local content.	Search by title/body/tags.
Favorites	View saved prayers.	Open or remove favorite.
Settings	Manage app preferences.	Language, reminder, font size, theme, about.

3.2 Hardware Interfaces

No special hardware interface is required for the MVP. The application shall not require or access GPS, camera, microphone, NFC, Bluetooth, biometric sensors, or background location. The only mobile hardware-related dependency is the device notification system for local reminders.

Hardware Feature	MVP Usage	Permission Required
Notification system	Daily prayer reminder using local notifications.	Notification permission where required by OS.
Storage	Local storage for preferences, favorites, and progress.	No special user permission required.
Camera	Not used.	None.
GPS / Location	Not used.	None.
Microphone	Not used.	None.
Biometrics	Not used because there is no authentication.	None.
Bluetooth / NFC	Not used.	None.

3.3 Software Interfaces

Software Interface	MVP Requirement
Flutter SDK	Core mobile framework.
Dart Runtime	Application logic and UI development.
Local Content Loader	Loads bundled JSON from assets/content.
Local Storage	Stores language, favorites, novena progress, Rosary progress, reminders, and settings.
Local Notifications Plugin	Schedules prayer reminders on-device.
Operating Systems	Android and iOS native services for notifications, local storage, and theming.
External Backend	Not required for MVP. Future optional HTTPS-based content update service may be added.
Database	No remote database for MVP. Local key-value storage or lightweight database may be used.
Apple/Google Pay	Not used.
HealthKit / Google Fit	Not used.
Contacts	Not used.

3.4 Communications Interfaces

The MVP shall not require network communication for core functionality. If the app includes links to external reading sources, privacy policy, support, or repository pages, these links shall open through the device browser using HTTPS URLs.

Communication Type	MVP Requirement
Internet	Not required for prayer content, Rosary, novenas,

	favorites, search, or settings.
HTTPS	Required only for external links and future content update service.
Push Notifications	Not used in MVP.
Local Notifications	Used for reminders without server dependency.
WebSockets	Not used.
SFTP / Email Routing	Not used.

4. Functional Requirements

4.1 System Workflows

4.1.1 First Launch and Language Selection Workflow

1. User installs and opens the app.
2. System checks local settings for selected language.
3. If no language is found, system displays Language Selection screen.
4. User selects English or Kiswahili.
5. System saves selected language locally.
6. System loads matching bundled content and navigates to Today screen.

4.1.2 Prayer Browsing Workflow

7. User opens Pray tab.
8. System loads category list from bundled category JSON.
9. User selects a category.
10. System filters prayers by selected language and category.
11. User taps a prayer.
12. System opens Prayer Detail screen and renders title, body, metadata, and favorite state.

4.1.3 Add New Prayer Content Workflow for Developers

13. Developer creates or edits a prayer JSON entry using the approved schema.
14. Developer places the entry in the correct language and category file.
15. Developer runs content validation tests.
16. Application loads the new content through the existing content loader.
17. No new UI screen is required unless a completely new content type is introduced.
18. QA verifies title, body, language, category, search visibility, and favorite behavior.

4.1.4 Rosary Workflow

19. User opens Rosary module.
20. System suggests mystery based on current local day.
21. User accepts suggested mystery or manually selects a mystery.
22. System initializes Rosary step sequence.
23. User uses Next/Previous controls to move through prayers.
24. System saves current progress locally after each step.

25. User can exit and later continue from saved progress.
26. User can restart Rosary, which clears current Rosary progress.

4.1.5 Novena Workflow

27. User opens Novenas tab.
28. System displays novenas available in selected language.
29. User selects a novena.
30. System shows description and Day 1-Day 9 structure.
31. User starts novena; system stores active novena and current day.
32. User reads day content and marks day completed.
33. System updates progress and makes continuation available on Today screen.
34. After Day 9 is completed, system marks novena as completed.

4.2 Feature Matrices

Module	Feature	Trigger	System Response	Priority
Onboarding	Language selection	First app launch	Display language options and save selected language.	Must
Today	Quick prayer access	App open or Today tab tap	Show daily prayer and shortcuts.	Must
Prayer Library	Category browsing	User opens Pray tab	Load categories and prayers from local content.	Must
Prayer Detail	Read prayer	User taps prayer	Render prayer detail and favorite action.	Must
Favorites	Save favorite	User taps favorite icon	Save/remove prayer ID locally.	Must
Search	Offline search	User enters keyword	Search selected-language local content.	Must
Rosary	Guided prayer	User starts Rosary	Generate step sequence and save progress.	Must
Novenas	9-day tracking	User starts novena	Save active novena and completed days.	Must
Reminder	Daily notification	User sets reminder	Schedule local notification.	Must
Settings	Change preferences	User updates setting	Persist preference locally and update UI.	Must
About	Content sources	User opens About	Show app, license, privacy, content disclaimer.	Should

4.3 Input/Output Handling

Input Area	Valid Input	Invalid Input Handling	Output
Language	en or sw language code.	Reject unsupported language and keep previous/default language.	App UI and content shown in selected language.

Search keyword	Text from 1 to 100 characters.	Ignore empty query or show recent/default results; trim whitespace.	List of matching prayers.
Prayer JSON id	Lowercase snake_case unique ID.	Validation fails during development/test.	Prayer appears in category/search.
Prayer body	Non-empty plain text or approved Markdown subset.	Validation error; content not accepted.	Rendered readable prayer detail.
Favorite action	Existing prayer ID.	If ID missing, show non-blocking error and do not save.	Favorite list updated.
Reminder time	Valid local time HH:mm.	Show validation message and require correction.	Local notification scheduled.
Novena day completion	Day integer 1-9.	Reject out-of-range day.	Progress updated.
Rosary step	Valid index in generated sequence.	Clamp to first/last valid step or reset if corrupted.	Current step rendered.

4.4 Detailed Functional Requirements

FR-001: Language Selection

Field	Requirement
Priority	Must
Description	Allow user to select preferred app language on first launch and later from Settings.
Inputs	Tap on English or Kiswahili option.
Processing / System behavior	Save language code locally; reload UI labels and content for selected language.
Outputs	Selected language persists and content uses selected language.
Validation and error handling	Unsupported language codes are ignored or rejected; missing translations use fallback rules.
Acceptance criteria	Language can be selected, changed, and persists after app restart.

FR-002: Today Screen

Field	Requirement
Priority	Must
Description	Provide a daily entry point for prayer.
Inputs	App open or Today tab selection.
Processing / System behavior	Load daily prayer suggestion, active novena status, Rosary shortcut, common prayers, and favorites shortcut.
Outputs	Today screen with quick actions.
Validation and error handling	If no active novena exists, hide or replace continue section.
Acceptance criteria	User can reach a prayer or Rosary within two taps.

FR-003: Prayer Categories

Field	Requirement
Priority	Must

Description	Organize prayers into content-driven categories.
Inputs	User opens Pray tab.
Processing / System behavior	Load categories from category JSON using selected language.
Outputs	Category list with icons/titles.
Validation and error handling	If category has no translated title, use fallback language or display ID safely.
Acceptance criteria	Categories render without hard-coded screen per category.

FR-004: Prayer List

Field	Requirement
Priority	Must
Description	Display prayers inside a selected category.
Inputs	User taps category.
Processing / System behavior	Filter prayers by category and language; sort by configured order/title.
Outputs	List of prayers with favorite state.
Validation and error handling	If no prayers found, show empty state.
Acceptance criteria	User can open a prayer detail screen.

FR-005: Prayer Detail

Field	Requirement
Priority	Must
Description	Render a full prayer from data.
Inputs	User taps prayer.
Processing / System behavior	Load title, body, source, category, and favorite state.
Outputs	Readable prayer content screen.
Validation and error handling	If content is missing/corrupt, show graceful error.
Acceptance criteria	Prayer is readable offline and can be favorited.

FR-006: Common Prayer Content

Field	Requirement
Priority	Must
Description	Include core Catholic prayers in the MVP.
Inputs	App content bundle loaded.
Processing / System behavior	Content loader indexes common prayers for selected language.
Outputs	Common prayer library available.
Validation and error handling	Missing translations do not crash app.
Acceptance criteria	At least agreed common prayers are available in English and priority Kiswahili translations.

FR-007: Favorites

Field	Requirement
Priority	Must
Description	Allow user to save prayers for quick access.
Inputs	User taps favorite/unfavorite.

Processing / System behavior	Store or remove prayer ID locally.
Outputs	Favorite icon/state and Favorites list updated.
Validation and error handling	Invalid prayer IDs are rejected.
Acceptance criteria	Favorite state persists after app restart.

FR-008: Search

Field	Requirement
Priority	Must
Description	Allow offline search across prayers.
Inputs	User enters keyword.
Processing / System behavior	Search title, body, tags, and category in selected language.
Outputs	Ranked list of results.
Validation and error handling	Empty query shows prompt; no matches shows empty result message.
Acceptance criteria	Search works offline and returns relevant results.

FR-009: Interactive Rosary

Field	Requirement
Priority	Must
Description	Guide users through Rosary steps.
Inputs	User starts Rosary.
Processing / System behavior	Suggest mystery by day; generate prayer sequence; allow next/previous; save progress.
Outputs	Current Rosary step and navigation controls.
Validation and error handling	If saved progress corrupt, reset with message.
Acceptance criteria	User can start, continue, navigate, and restart Rosary offline.

FR-010: Novenas

Field	Requirement
Priority	Must
Description	Provide structured 9-day novenas and progress tracking.
Inputs	User starts or continues novena.
Processing / System behavior	Load novena content; save active day and completed days locally.
Outputs	Novena day content and progress state.
Validation and error handling	Reject invalid day values.
Acceptance criteria	User can complete Day 1-Day 9 and continue from Today screen.

FR-011: Daily Reminder

Field	Requirement
Priority	Must
Description	Allow user to schedule a local prayer reminder.
Inputs	User enables reminder and selects time.
Processing / System behavior	Schedule local notification on device.

Outputs	Notification shown at selected time.
Validation and error handling	Invalid time rejected; permission denial handled gracefully.
Acceptance criteria	User can enable, change, and disable reminder.

FR-012: Offline Mode

Field	Requirement
Priority	Must
Description	Ensure core app features work without internet.
Inputs	User opens app offline.
Processing / System behavior	Use bundled content and local storage only.
Outputs	Prayer, Rosary, novenas, favorites, search, settings available.
Validation and error handling	External links show unavailable/offline behavior if needed.
Acceptance criteria	Core MVP remains usable in airplane mode.

FR-013: Settings

Field	Requirement
Priority	Must
Description	Allow user to manage preferences.
Inputs	User opens Settings.
Processing / System behavior	Read/update local preferences for language, theme, font, reminder.
Outputs	Updated preferences applied.
Validation and error handling	Invalid settings ignored or reset to defaults.
Acceptance criteria	Settings persist after app restart.

FR-014: Content Attribution

Field	Requirement
Priority	Should
Description	Show source and legal notes for prayer content.
Inputs	User opens About or prayer source info.
Processing / System behavior	Render source metadata and content disclaimer.
Outputs	Source/attribution information displayed.
Validation and error handling	If source field missing, hide source line.
Acceptance criteria	About screen includes content and license notes.

FR-015: Content Extensibility

Field	Requirement
Priority	Must
Description	Allow new prayers to be added by content file changes, not UI changes.
Inputs	Developer adds valid JSON entry.
Processing / System behavior	Content loader indexes item based on schema and category.
Outputs	New prayer appears in app.
Validation and error handling	Validation fails for missing required fields.

Acceptance criteria	Adding a prayer does not require a new screen.
---------------------	--

4.5 User Stories and Use Cases

Use Case ID	Name	Preconditions	Primary Success Path	Alternate Path	Postconditions
UC-001	Choose language	New user has installed app.	User opens app, selects Kiswahili, system saves setting and opens Today screen.	User closes app before selecting language; system asks again on next launch.	Language is saved and app content attempts to display in Kiswahili.
UC-002	Read a common prayer	User has selected language.	User opens Pray, taps Common Prayers, taps Hail Mary, reads prayer.	Prayer translation is missing; app shows fallback or unavailable message.	Prayer detail screen is displayed.
UC-003	Favorite a prayer	Prayer detail is open.	User taps Favorite icon; system saves prayer ID; user opens Favorites and sees the prayer.	Storage write fails; app shows non-blocking error.	Favorite persists locally.
UC-004	Pray the Rosary	User opens Rosary module.	System suggests mystery by day; user starts; user taps Next through sequence.	User exits early; app saves progress.	Rosary progress is stored or completed.
UC-005	Start and continue novena	User opens Novenas.	User selects Divine Mercy Novena, starts Day 1, marks complete, later continues Day 2 from Today screen.	User completes days out of order; app may allow manual day selection but stores completed status correctly.	Active novena progress is available.
UC-006	Use app offline	User has installed app.	User turns off internet, opens app, searches prayers, reads prayer, continues novena.	User taps external link; app shows offline/unavailable message.	Core content remains accessible.
UC-007	Add new prayer as developer	Developer has repository access.	Developer adds JSON prayer, runs validator/tests, builds app, prayer appears in category/search.	Schema validation fails; developer corrects data.	New prayer appears without UI code changes.

5. Non-Functional Requirements (System Attributes)

5.1 Performance

Metric	Requirement
Cold startup	Target under 2 seconds on modern devices and under 3 seconds on low-end supported Android devices.
Prayer detail load	Under 1 second from tap to rendered content for bundled content.
Search response	Under 1 second for MVP content size.
Memory usage	Should remain lightweight; avoid loading unnecessary large assets. Target under 150 MB runtime memory on common devices.
Battery	No background network polling in MVP. Local notifications should not cause continuous background processing.
App size	Keep text-first MVP lightweight; avoid audio/video assets in MVP.

5.2 Reliability and Availability

Attribute	Requirement
Availability	Because core features are local, functional availability target for core prayer features is 99.9% excluding OS/device failures.
Fault tolerance	Corrupt user progress should not crash app; reset affected progress with a friendly message.
Offline reliability	All core MVP features shall work without internet.
MTBF goal	No crash in normal core flows during a 2-hour exploratory QA session.
Backup	No cloud backup in MVP. Local data is device-specific.

5.3 Security and Privacy

Security Area	Requirement
Authentication	No authentication service in MVP. No biometric login, Face ID, Touch ID, Android Fingerprint, Apple Sign-In, Google Sign-In, or social login.
Data collection	Do not collect sensitive personal data in MVP.
Local data	Favorites, settings, reminder preferences, and progress stored locally. Since data is low sensitivity, OS sandboxing is acceptable; use encrypted storage only if future sensitive data is added.
Transport security	Any future network calls must use HTTPS. SSL pinning is not required for MVP because no backend API is used; reconsider if sensitive APIs are introduced.
Permissions	Request only permissions required for notifications. Do not request location, camera, microphone, contacts, or health permissions.
Privacy policy	A privacy policy/support link shall be prepared for store submission, even if the app collects minimal or no personal data.
RBAC	Not applicable in MVP because there are no user roles

	or admin module in the app.
--	-----------------------------

5.4 Maintainability and Scalability

- The codebase shall separate UI, domain logic, content loading, and local storage.
- The app shall use reusable components for prayer list, prayer detail, Rosary step, and novena day rendering.
- Adding a new prayer shall require content changes only, not screen code changes.
- Content validation scripts/tests should verify required fields, language codes, IDs, and duplicate IDs.
- Future remote content support shall be possible by adding a repository/service layer behind the same content interface.
- The project should include README documentation for developers and content contributors.

5.5 Portability

- The application shall be portable across Android and iOS using Flutter.
- Platform-specific code shall be isolated behind services or plugins.
- No feature shall rely on Android-only behavior unless iOS fallback is defined.
- The content model shall be platform-independent JSON.
- Future web/desktop ports are not MVP goals but should not be blocked by poor content architecture.

5.6 Mobile Non-Functional Requirements

Mobile Area	Requirement
Backgrounding	When app goes to background, save current Rosary and novena progress immediately or on next lifecycle event.
Sudden interruptions	Incoming calls, OS app switching, and low-memory events must not lose completed novena day or current Rosary step where possible.
Notification handling	Tapping a local reminder should open the app, preferably to Today screen or active novena when feasible.
Slow networks	Network is not needed for core features. External links may fail gracefully.
Screen scaling	Support dynamic font scaling without clipping critical controls.
Battery	No continuous background tasks or polling in MVP.

5.7 Compliance and Deployment

5.7.1 Store Guidelines

- The iOS build shall comply with Apple App Store Review Guidelines, including safety, performance, business, design, and legal review areas.
- The Android build shall comply with Google Play Developer Program Policies and Google Play target API level requirements at release time.

- All external links in the app shall be functional before store submission.
- The app shall provide support contact information and a privacy policy link for store submission.
- The app shall avoid requesting unnecessary permissions to reduce review risk and improve user trust.
- Content ownership and source/permission status shall be reviewed before publishing.

5.7.2 Distribution

Distribution Stage	Requirement
Internal QA	Debug/release candidate builds distributed to internal testers.
Android Beta	Use Google Play Internal Testing, Closed Testing, or equivalent beta channel.
iOS Beta	Use TestFlight if iOS release is included.
Production Android	Publish signed Android App Bundle to Google Play.
Production iOS	Publish via App Store Connect after TestFlight validation.
Versioning	Use semantic app versioning such as 0.1.0 for first MVP beta and 1.0.0 for first stable public release.

6. Supporting Information

6.1 Data Models

6.1.1 Data Dictionary

Entity	Field	Type	Required	Description
Prayer	id	string	Yes	Unique lowercase snake_case identifier.
Prayer	type	string	Yes	Content type, e.g., prayer.
Prayer	category	string	Yes	Category ID.
Prayer	language	string	Yes	Language code such as en or sw.
Prayer	title	string	Yes	Localized prayer title.
Prayer	description	string	No	Short optional summary.
Prayer	body	string	Yes	Prayer text; may support approved Markdown subset.
Prayer	tags	array<string>	No	Search and grouping tags.
Prayer	source	string	No	Content source or attribution.
Prayer	version	integer	No	Content version.
Prayer	last_updated	date	No	Date content was last updated.
Category	id	string	Yes	Unique category ID.

Category	title	map	Yes	Localized title map by language code.
Category	sort_order	integer	Yes	Display ordering.
Novena	id	string	Yes	Unique novena ID.
Novena	days	array	Yes	Nine novena day objects.
UserPreference	selected_language	string	Yes	Language code saved locally.
UserPreference	font_size	number	No	Reading font size preference.
Favorite	prayer_id	string	Yes	Prayer ID saved as favorite.
NovenaProgress	novena_id	string	Yes	Active novena ID.
NovenaProgress	completed_days	array<int>	Yes	Completed novena days.
RosaryProgress	mystery_id	string	No	Selected mystery.
RosaryProgress	step_index	integer	No	Current Rosary step index.

6.1.2 Prayer JSON Schema Example

```
{
  "id": "hail_mary",
  "type": "prayer",
  "category": "common_prayers",
  "language": "en",
  "title": "Hail Mary",
  "description": "Traditional Marian prayer.",
  "body": "Hail Mary, full of grace...",
  "tags": ["marian", "daily", "rosary"],
  "source": "Traditional Catholic prayer",
  "version": 1,
  "last_updated": "2026-06-02",
  "is_offline_available": true
}
```

6.1.3 Category JSON Schema Example

```
{
  "id": "common_prayers",
  "title": { "en": "Common Prayers", "sw": "Sala za Kawaida" },
  "description": {
    "en": "Essential Catholic prayers for daily use.",
    "sw": "Sala muhimu za Kikatoliki kwa matumizi ya kila siku."
  },
  "sort_order": 1,
  "icon": "hands_prayer"
```

}

6.2 Analysis Models

6.2.1 High-Level Data Flow Diagram

User -> Flutter UI -> State Manager -> Content Repository -> Local JSON Assets

User -> Flutter UI -> State Manager -> Local Storage -> Preferences/Favorites/Progress

User -> Settings -> Local Notifications Service -> OS Notification System

6.2.2 Content Loading State Model

State	Description	Next State
Uninitialized	App has started but settings/content not loaded.	LoadingSettings
LoadingSettings	System reads selected language and user preferences.	LoadingContent
LoadingContent	System loads bundled JSON for selected language.	Ready or Error
Ready	Content is indexed and UI can render.	RefreshingContent in future remote update scenario
Error	Content loading failed.	FallbackContent or SafeErrorScreen
FallbackContent	System uses default English or bundled safe content.	Ready

6.2.3 Rosary Progress State Model

State	Trigger	System Behavior
Not Started	User has no active Rosary.	Show Start Rosary.
In Progress	User starts and advances steps.	Save mystery and step index.
Paused	User exits app or leaves Rosary screen.	Keep progress locally.
Completed	User reaches final step.	Mark complete and offer restart.
Reset	User taps Restart.	Clear progress and start at first step.

6.3 Acceptance Criteria

Area	Acceptance Criteria
Onboarding	First-time user can select English or Kiswahili and reach Today screen.
Offline	With internet disabled, user can open prayers, search, use Rosary, use novenas, and view favorites.
Prayer Content	Common prayers are loaded from JSON, not hard-coded in UI screen files.
Add New Prayer	Developer can add a new valid prayer JSON entry and see it in category/search without creating a new screen.
Favorites	Favorites can be added, removed, displayed, and persisted after app restart.
Search	Search finds prayers by title and relevant content in selected language.
Rosary	User can start, navigate next/previous, exit, and continue Rosary.
Novenas	User can start a novena, mark days complete, and continue active novena.

Reminder	User can schedule, change, and disable daily reminder.
Settings	User can change language, font size, theme preference, and reminder preference.
Permissions	App does not request camera, microphone, contacts, location, NFC, Bluetooth, or biometric permissions.
Store Readiness	App has functional support link, privacy policy link, content attribution, and minimal permission declaration.

6.4 Traceability Matrix

Business Requirement	Functional Requirement(s)	Acceptance / Test Area
BR-001: Help users pray daily.	FR-002, FR-005, FR-006, FR-011	Today screen, prayer detail, reminder tests.
BR-002: Provide essential Catholic prayers.	FR-003, FR-004, FR-005, FR-006	Category and prayer content tests.
BR-003: Support Rosary devotion.	FR-009	Rosary workflow tests.
BR-004: Support novena habit and retention.	FR-010, FR-011	Novena progress and reminder tests.
BR-005: Work offline.	FR-012	Airplane mode regression tests.
BR-006: Support English and Kiswahili.	FR-001, FR-003, FR-006, FR-013	Localization tests.
BR-007: Enable quick access to preferred prayers.	FR-007, FR-008	Favorites and search tests.
BR-008: Enable future prayer additions without screen hard-coding.	FR-015	Content schema validation and dynamic rendering test.
BR-009: Avoid unnecessary user data collection.	FR-013 and NFR Security	Permission/privacy review.
BR-010: Prepare for app store release.	FR-014 and NFR Compliance	Store checklist and policy review.

Appendix A. Recommended Content File Structure

```

assets/
content/
  categories.json
  prayers/
    en/
      common_prayers.json
      marian_prayers.json
      mass_prayers.json
    sw/
      common_prayers.json
      marian_prayers.json
      mass_prayers.json
  rosary/

```

en/rosary.json
sw/rosary.json
novenas/
en/divine_mercy.json
sw/divine_mercy.json

Appendix B. Out-of-Scope Backlog

Future Feature	Reason Deferred
Backend / CMS	Not needed for MVP; adds hosting, admin, security, and sync complexity.
Remote content updates	Useful later after content model stabilizes.
User accounts and cloud sync	Not required for offline-first MVP.
Audio prayers	High production and storage cost; validate text MVP first.
Full Bible	Licensing and data complexity.
Daily Mass readings integration	Potential copyright/licensing complexity; can be added with authorized sources later.
Community features	Moderation burden and safety concerns.
AI spiritual assistant	Pastoral, theological, and safety risk; not needed for MVP.
Donations / subscriptions	Not needed before user value and retention are validated.